3D-PRINTABLE · ARDUINO NANO 33 IOT · MQTT

# Open Gauges: Wind Direction Gauge

## Stepper edition

A 205 mm wall-mounted dial that shows live wind direction from any weather station publishing an MQTT data feed. A 28BYJ-48 stepper turns the pointer, a hall sensor finds North on start-up, ensuring accuracy and an LED strip lights the dial from behind. Its made to be as simple to print and make as possible, there as a few wires to connect but they all line up logically on the breadboard and if you take it part by part it can be wired up in under 10 minutes.




| | |
|---|---|
| Prints on a 220 mm-class bed. Largest part 204.8 mm (bezel). Five parts, no supports. | Self-homing. Hall sensor and magnet set North, and the sketch measures steps per revolution itself. |
| Screw-on bezel. M200 × 4 thread with print clearance, 190 mm acrylic window. | Clip-in parts. Dial, LED strip, boards and breadboard all clip in. One screw hangs it on the wall. |

Design and build: Andy Hudson-Smith · digitalurban.org

HOW IT WORKS

# From a weather feed to a moving pointer

Our weather station from the Connected Environments Lab at the Centre for Advanced Spatial Analysis, University College London publishes the wind direction (in degrees) to an MQTT broker. We use this feed via an Arduino Nano 33 IoT which subscribes over Wi-Fi and turns the 28BYJ-48 stepper by the shortest path to that bearing. The pointer sits on a small printed **magnet carrier** on the motor shaft. A 5 mm magnet in the carrier passes over a KY-003 hall sensor at North, so the gauge can find North on start-up and re-zero itself every six hours (to counter any drift).

A WS2812B LED strip clips into a channel under the dial, above all the boards and wiring, so it lights the dial evenly without casting wire shadows.



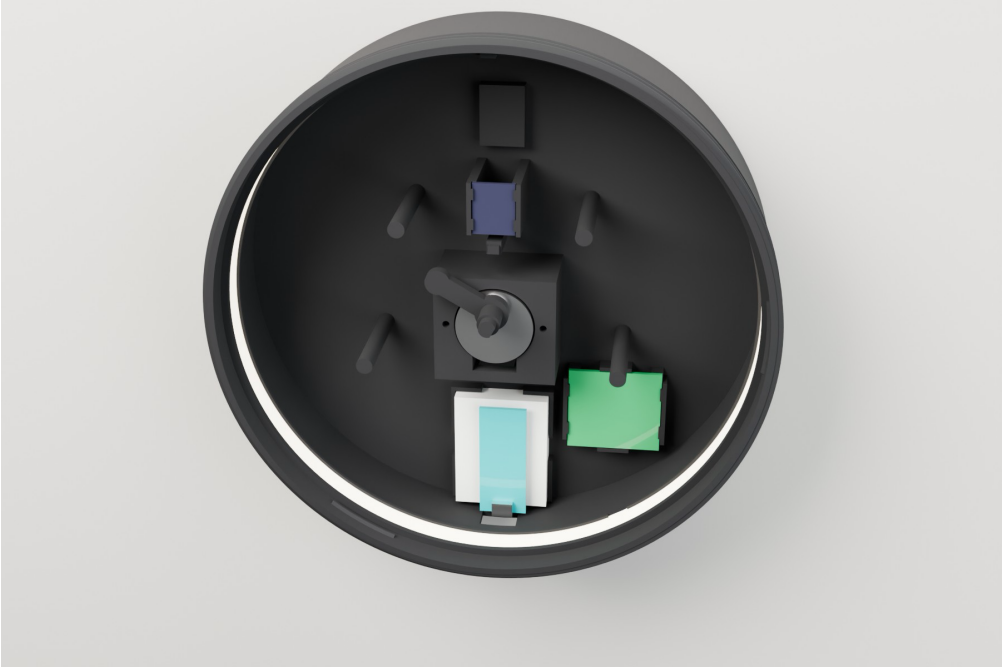

*Exploded view, back to front: case with motor, ULN2003, breadboard + Nano and LED channel; magnet carrier; dial; pointer; acrylic window; screw-on bezel.*

INSIDE THE CASE

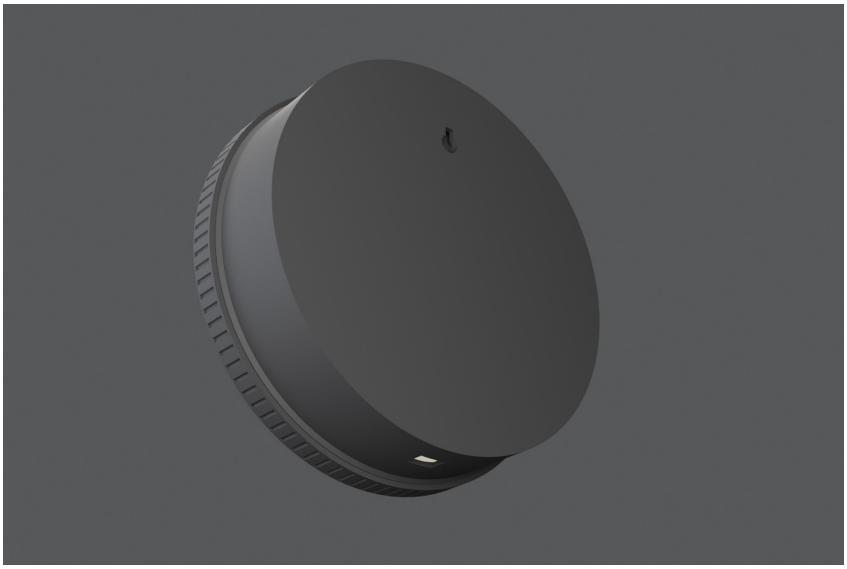
# Where everything goes

Every part has its own printed seat, so nothing needs glue except the magnet (although this can also be held using an additional magnet on top). The 28BYJ-48 sits in the centre block with the magnet carrier on its shaft. The KY-003 hall sensor is in the cradle above it, under the magnet's path. The ULN2003 driver snaps in to the right, and the mini breadboard with the Nano 33 IoT sits in a tray at the bottom: its side tabs drop into two notches and the adhesive back holds it down. The Nano's micro-USB end faces a 14 × 10 mm opening in the bottom wall.

The LED strip clips into a channel round the inside wall, just under the dial, so it sits above every board and wire. A gap in the channel's lip at 5 o'clock lets the strip's wires drop down to the breadboard. On the back, a single centred keyhole hangs the gauge on one screw.



*From the front, dial removed.*



*From below and behind: keyhole and USB opening.*

WHAT YOU NEED

# Parts list

## Printed parts

| Part | File | Print orientation | Size (mm) |
|---|---|---|---|
| Case | WG-01_Case_back-down | Back down | 199.7 × 199.7 × 53.3 |
| Dial: raised (choose one) | WG-02_Dial_face-up_M600 | Face up, colour change at 2.2 mm | 190.4 × 3.0 |
| Dial: flat two-colour (choose one) | WG-02F_Flat_dial_2colour_Bambu (.3mf) | Face down, AMS or manual swaps | 190.4 × 2.0 |
| Bezel | WG-03_Bezel_front-down | Front down | 204.8 × 15.3 |
| Pointer | WG-04_Pointer | Underside down | 12 × 117 × 4.4 |
| Magnet carrier | WG-05_Magnet_carrier | Underside down | 12 × 32.5 × 17.2 |

The bezel needs at least 205 mm in one direction. It fits a Prusa MK4 (250 × 210) and Bambu X1, P1 and A1 (256 × 256), but not an A1 mini or Prusa MINI.

## Bought parts

| Item | Notes |
|---|---|
| Arduino Nano 33 IoT (with headers) | Wi-Fi via the WiFiNINA library. 3.3 V logic. |
| Mini breadboard, 170 tie-points (ELEGOO or similar) | About 46 × 35 × 9.4 mm. Its side tabs drop into the tray notches; peel the adhesive back to stick it down. |
| 28BYJ-48 stepper (5 V) + ULN2003 driver board | The 5 V version, not the 12 V one. |
| KY-003 hall sensor module (A3144) | Powered from 5 V. |
| Neodymium magnet, 5 mm diameter, up to 3 mm thick | Glued into the magnet carrier. |
| WS2812B LED strip, 10 mm wide, up to 2.2 mm thick | About 0.6 m round the inside of the case. Bare or IP30 strip, not the thick silicone-sleeved type. |
| Clear acrylic disc, Ø190 × 3 mm | Laser cut from Acrylic_190mm_x3_disc.dxf, or cut from 3 mm A4 sheet and sand to 189.5–190.2 mm. |
| 2 × M3 × 6 pan-head screws | Motor ears; they self-tap into the printed pilot holes. |
| Jumper wires, micro-USB cable, 5 V USB supply | A 2 A supply gives the LEDs headroom. |
| 1 × wall screw | Head up to 9.5 mm across and 3.5 mm thick. |

PRINTING

# Print settings

| Setting | Value |
|---|---|
| Material | PLA (black case, bezel and carrier; white dial; red pointer looks good) |
| Layer height | 0.2 mm |
| Walls / infill | 3 walls, 15 % infill |
| Supports | None: every part is already oriented in its file |
| Brim | Not needed. Clean the sheet with IPA for the large parts. |

## Part notes

**Case.** The LED channel's top cap is a short 2.8 mm flat overhang off the wall; it prints without supports but may sag slightly underneath. The USB opening and keyhole are bridged.

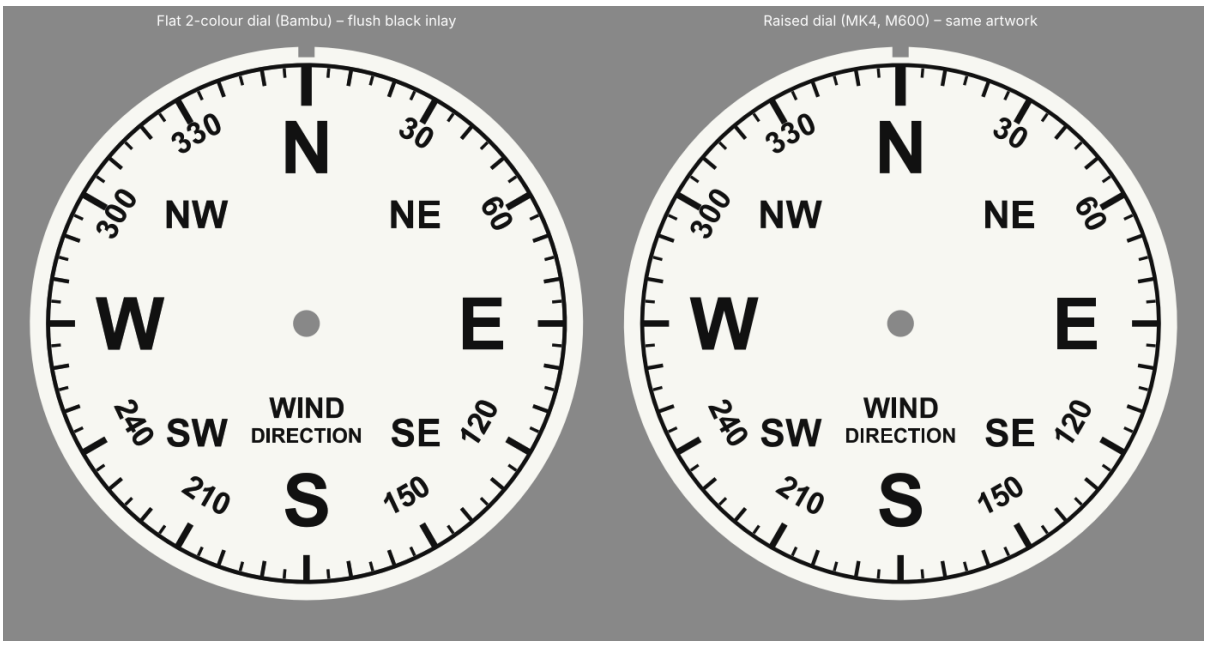
**Bezel.** The M200 × 4 thread has 0.2 mm clearance built in. Print it in the same filament as the case so the shrinkage matches.

**Pointer and carrier.** Both are designed to push on until they stop: the carrier stops on the end of the motor shaft, the pointer on the top of the spigot. That keeps the pointer 1 mm above the dial and 1 mm below the acrylic.

## Dial options

**Raised dial (single-extruder).** Print WG-02 face up in white and add a filament change (M600) at the 2.2 mm layer: the marks stand 1 mm proud in black.

**Flat two-colour dial.** WG-02F has the black marks inlaid flush, 0.6 mm deep, and prints face down so the face is dead flat. On a Bambu with AMS, open the .3mf and set *Dial white* to white and *Marks black* to black. On a Prusa MK4 without an MMU you can do it with three or four manual swaps: set up a two-extruder "Single Extruder Multi Material" profile with `M600` as the tool-change G-code and a small wipe tower. Multicolour printers can of course do this with ease, our dial was printed on a Bambu machine.




*The two dials share the same artwork. Left: flat two-colour inlay. Right: raised marks.*

ASSEMBLY

# Putting it together

Test the electronics on the bench before you close the case: it is much easier to fix a wire with the dial out.

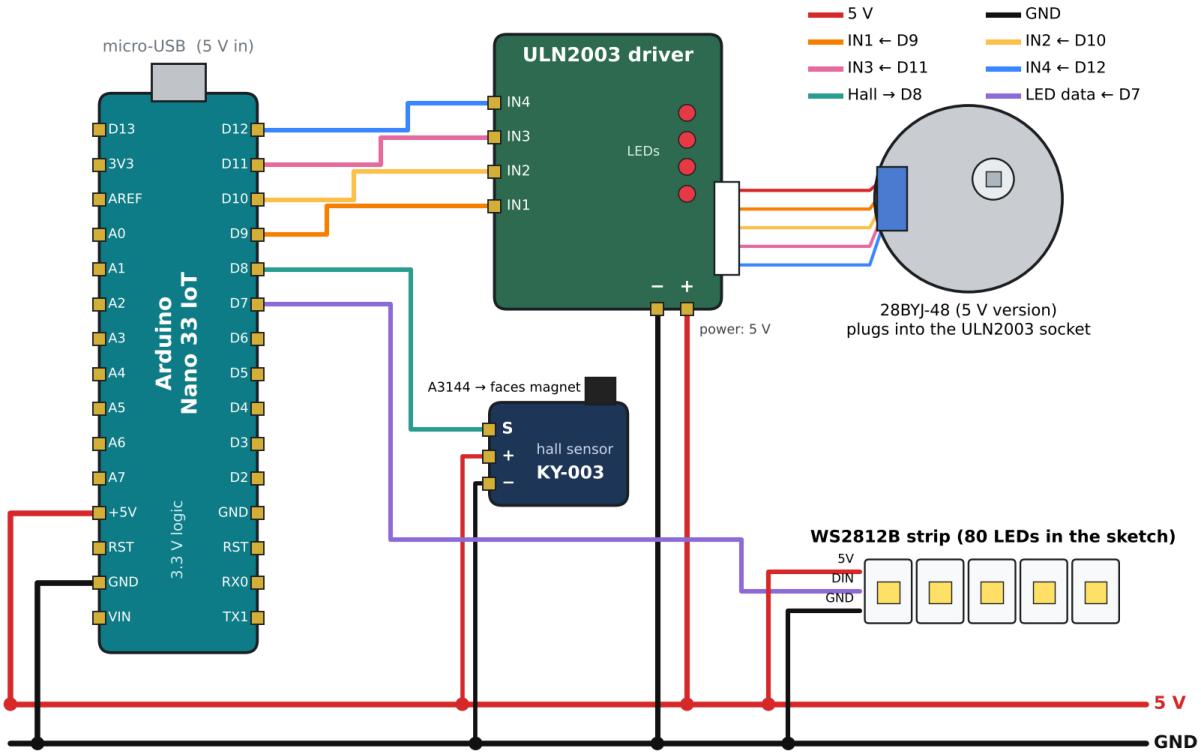
1. **Test-fit first.** Before adding electronics, check the dial clicks into the case and the bezel screws on.
2. **Motor.** Drop the 28BYJ-48 into its pocket from the front, wire cover pointing down, and fix the ears with two M3 × 6 screws.
3. **Driver.** Snap the ULN2003 board into its cradle to the right of the motor and plug in the motor cable.
4. **Breadboard and Nano.** Press the mini breadboard into the tray at the bottom: its side tabs go into the two notches. Peel the adhesive backing first to stick it down. Plug the Nano in with its micro-USB end against the case wall, then check a cable reaches through the opening.
5. **Hall sensor.** Bend the A3144 so it lies flat, face up, on the small shelf below the KY-003 cradle, and snap the board in.
6. **Magnet carrier.** Power up and hold the magnet over the sensor: the KY-003 LED should light. If not, flip the magnet. Glue it into the carrier pocket the right way up, or put another magnet on top to hold it, then push the carrier onto the shaft until it stops.
7. **LED strip.** Starting at the wire gap at 5 o’clock, drop the strip’s lower edge behind the lip and press the top edge under the cap, LEDs facing inward. Work round the case; no adhesive needed.
8. **Wire up** as shown on the next page, upload the sketch and let it home and sweep to South and back.
9. **Dial.** Line up the notch with the rib at 12 o’clock, tuck it under two of the snap bumps and press the third side down until it clicks.
10. **Pointer.** Push it onto the spigot until it stops. It only fits one way.
11. **Window and bezel.** Peel the film off the acrylic, clip it into the bezel from behind, and screw the bezel on hand-tight until it seats on the case rim.
12. **Hang it.** The single keyhole puts the screw 67 mm directly above the dial centre. Leave the screw head about 3 mm out from the wall.

ELECTRONICS

# Wiring the Nano 33 IoT

### WindGauge (stepper edition): wiring for the Arduino Nano 33 IoT

Pins match the sketch: motor D9–D12, hall sensor D8, LEDs D7. Everything runs from the Nano's +5V pin (USB power), all grounds common.




| Nano 33 IoT | Connects to |
|---|---|
| D9 / D10 / D11 / D12 | ULN2003 IN1 / IN2 / IN3 / IN4 |
| D8 | KY-003 S (signal) |
| D7 | WS2812B DIN (data in) |
| +5V | ULN2003 +, KY-003 +, LED strip 5V |
| GND | ULN2003 −, KY-003 −, LED strip GND |

**As built, no extra components are needed:** the KY-003 and the strip connect straight to the pins. Two optional extras if you want extra margin: the KY-003 board pulls its output up to 5 V while the Nano's pins are rated for 3.3 V, so a 10 kΩ / 20 kΩ divider on S keeps it within spec; and a 330 Ω resistor on LED data plus a 1000 µF capacitor across the strip's power is standard WS2812B practice.

FIRMWARE

# The Arduino sketch

The full sketch is printed at the end of this guide and included as `WindGauge_Nano33IoT.ino`.

## Set up

1. In the Arduino IDE, install the board package **Arduino SAMD Boards** and select **Arduino Nano 33 IoT**.
2. Install the libraries **AccelStepper**, **WiFiNINA**, **PubSubClient** and **Adafruit NeoPixel** from the Library Manager.
3. Edit the settings at the top of the sketch (below), then upload with the Serial Monitor open at 115200 baud.

## What to change

| Setting | What it does |
|---|---|
| `ssid`, `password` | Your Wi-Fi network. |
| `mqtt_server`, `mqtt_port` | Your MQTT broker. |
| `wind_topic` | The topic your station publishes wind direction to. The payload is a plain number of degrees, e.g. `247`. |
| `neoNumPixels` | Number of LEDs on your strip (80 in the sketch). |
| `neoBrightness` | LED brightness, 0–255. Turn it down if the board resets or the USB supply struggles. |
| `northOffsetDeg` | Fine-tunes North (see below). |
| `client.connect("NanoIoTStepperClient")` | Give each gauge a unique client ID if you run more than one. |

## What happens at start-up

The LEDs come on, then the motor turns until the magnet reaches the hall sensor. After a 5-second pause it makes one full turn to measure the motor's real steps per revolution, so any 28BYJ-48 calibrates itself. It then moves to true North, sweeps to South and back as a self-test, and connects to Wi-Fi and MQTT. From then on it follows every new reading by the shortest route, and re-homes on the magnet every six hours to cancel any drift.

## Setting North exactly

Watch the pointer when the Serial Monitor prints *"At true North. Check the pointer against the N mark."* If it stops short of N (anticlockwise of it), increase `northOffsetDeg`; if it overshoots, decrease it. It will do a first sweep to the magnetic stop point and then correct to the true North. Negative values are fine. If the sensor reads HIGH on the magnet rather than LOW, change `atNorth()` as noted in the sketch.

WHEN THINGS GO WRONG

# Troubleshooting

| Symptom | Likely cause and fix |
|---|---|
| Motor keeps turning at start-up and never stops | The hall sensor isn't triggering. Check the KY-003 LED lights with the magnet over it; flip the magnet if not. The gap should be about 1.5 mm. |
| ULN2003 LEDs and strip stay dark | No 5 V on the Nano's +5V pin. Current boards supply it from USB out of the box; on some older Nano 33 IoT boards the pin only goes live once the two VUSB pads on the underside are bridged with solder. |
| Motor buzzes but doesn't turn | IN1–IN4 swapped. Wire D9–D12 to IN1–IN4 in order; the unusual pin order in the AccelStepper line is correct for the 28BYJ-48. |
| Board resets when the LEDs light | Too much current for USB. Lower `neoBrightness` or use a stronger supply. |
| Pointer rubs the dial or acrylic | Push the carrier fully onto the shaft and the pointer fully onto the spigot; both have hard stops. |
| Breadboard won't drop into the tray | Its side tabs don't line up with the notches. The notch width is a parameter (`bb_notch_w`) in the Fusion model. |

## Files in the download

| File | What it is |
|---|---|
| WG-01 … WG-05 (.stl / .3mf) | The printed parts, already oriented for printing. |
| WG-02F_Flat_dial_2colour_Bambu.3mf | Flat two-colour dial with both parts in place (white body, black inlay). |
| WG-02F_…_WHITE / _BLACK_face-down.stl | The same two dial parts as separate STLs for other slicers. |
| Acrylic_190mm_x3_disc.dxf | Cutting file for the acrylic window. |
| STEP files and the Fusion .f3d | The full parametric model. Breadboard tray, notches and clearances are user parameters. |
| WindGauge_Nano33IoT.ino | The Arduino sketch. |
| Wiring diagram (.png / .pdf) | The diagram from this guide at full size. |

**APPENDIX**

# Full sketch: WindGauge_Nano33IoT.ino

```
#include <AccelStepper.h>
#include <WiFiNINA.h>
#include <PubSubClient.h>
#include <Adafruit_NeoPixel.h>

// Define motor control pins for 28BYJ-48
#define motorPin1  9
#define motorPin2  10
#define motorPin3  11
#define motorPin4  12

// Define hall sensor pin (magnet near North for homing)
#define hallSensorPin 8

// NeoPixel configuration
#define neoPin 7                      // Pin where the NeoPixel is connected
#define neoNumPixels 80               // Number of NeoPixels
#define neoBrightness    255           // Brightness level (0-255)
#define neoType NEO_GRB + NEO_KHZ800  // NeoPixel strip type (e.g., NEO_GRB)

// Create a NeoPixel object
Adafruit_NeoPixel strip = Adafruit_NeoPixel(neoNumPixels, neoPin, neoType);

// Wi-Fi credentials
const char* ssid     = "Your WIFI SSID";
const char* password = "Your Password";

// MQTT broker details
const char* mqtt_server = "mqtt.cetools.org";
const int   mqtt_port   = 1883;
const char* wind_topic  = "personal/ucfnaps/downhamweather/windDir";

// ------------------------------------------------------------------------
// NORTH OFFSET (degrees)
// The hall sensor triggers slightly BEFORE true North because of where the
// magnet sits. This is how far the pointer must travel FORWARD (clockwise)
// from the magnet trigger point to land exactly on the N mark.
//   Pointer stops short of N (anticlockwise of it)  -> increase this value
//   Pointer overshoots N (clockwise of it)           -> decrease this value
// Negative values are fine if the magnet triggers after North.
// ------------------------------------------------------------------------
const float northOffsetDeg = 6.0;

// Define stepper parameters
// These are RUNTIME variables, not fixed defines: a full magnet->magnet sweep at
// startup measures the real step count, so any motor (2048, 4096, etc.) self-calibrates.
long  stepsPerRevolution = 2048;                      // Default; overwritten by calibration
float stepsPerDegree     = stepsPerRevolution / 360.0;  // Recomputed after calibration

// Auto re-home: periodically re-zero on the North magnet to cancel any drift.
const unsigned long rehomeIntervalMs = 6UL * 60UL * 60UL * 1000UL; // every 6 hours
unsigned long lastRehomeMs = 0;

// Homing / calibration rotation speed (steps/sec). The 28BYJ-48 stays reliable up
// to ~500; back this off if it stalls or misses the magnet.
const float homingSpeed = 300.0;

// Create AccelStepper instance
AccelStepper stepper(AccelStepper::HALF4WIRE, motorPin1, motorPin3, motorPin2, motorPin4);

// Wi-Fi and MQTT clients
WiFiClient espClient;
PubSubClient client(espClient);

// Global variable to store the target wind direction (degrees, 0-360)
float targetWindDir = 0.0;
float lastTargetWindDir = -1.0; // To track changes in wind direction

void printWiFiStatus() {
  switch (WiFi.status()) {
    case WL_IDLE_STATUS:     Serial.println("Wi-Fi status: Idle"); break;
    case WL_NO_SSID_AVAIL:   Serial.println("Wi-Fi status: SSID not available"); break;
    case WL_CONNECT_FAILED:  Serial.println("Wi-Fi status: Connection failed"); break;
    case WL_DISCONNECTED:    Serial.println("Wi-Fi status: Disconnected"); break;
    case WL_CONNECTED:       Serial.println("Wi-Fi status: Connected"); break;
    default:                 Serial.println("Wi-Fi status: Unknown"); break;
  }
}

void connectToWiFi() {
  Serial.print("Connecting to Wi-Fi: ");
  Serial.println(ssid);

  WiFi.begin(ssid, password);

  int retryCount = 0;
  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.print(".");
    printWiFiStatus();
```

```
    retryCount++;

    if (retryCount > 30) { // Timeout after 30 seconds
      Serial.println("\nWi-Fi connection failed! Restarting...");
      NVIC_SystemReset();
    }
  }

  Serial.println("\nWi-Fi connected.");
  Serial.print("IP address: ");
  Serial.println(WiFi.localIP());
}

void reconnectToMQTT() {
  while (!client.connected()) {
    Serial.print("Connecting to MQTT broker...");

    if (client.connect("NanoIoTStepperClient")) {
      Serial.println("connected.");
      client.subscribe(wind_topic);
    } else {
      Serial.print("failed, rc=");
      Serial.print(client.state());
      Serial.println(" retrying in 5 seconds...");
      delay(5000);
    }
  }
}

void mqttCallback(char* topic, byte* payload, unsigned int length) {
  String message;
  for (unsigned int i = 0; i < length; i++) {
    message += (char)payload[i];
  }

  Serial.print("Raw MQTT message: ");
  Serial.println(message);

  if (String(topic) == wind_topic) {
    targetWindDir = message.toFloat();
    Serial.print("Updated target wind direction (deg): ");
    Serial.println(targetWindDir);
  }
}

void moveStepperToWindDirection() {
  if (targetWindDir == lastTargetWindDir) {
    return;
  }

  // Normalise incoming direction to 0-360
  float dir = fmod(targetWindDir, 360.0);
  if (dir < 0) dir += 360.0;

  // Step 0 is now TRUE North (offset already applied when homing)
  long targetStep = lround(dir * stepsPerDegree);

  // --- SHORTEST-PATH ROTATION ---
  long current = stepper.currentPosition();
  long currentMod = ((current % stepsPerRevolution) + stepsPerRevolution) % stepsPerRevolution;

  long delta = targetStep - currentMod;
  if (delta >  stepsPerRevolution / 2) delta -= stepsPerRevolution;
  if (delta < -stepsPerRevolution / 2) delta += stepsPerRevolution;
  // ------------------------------

  Serial.print("Setting new target position (steps): ");
  Serial.println(current + delta);

  stepper.moveTo(current + delta);
  lastTargetWindDir = targetWindDir;
}

// True while the magnet is under the hall sensor.
// If yours reads HIGH on the magnet, change to: return digitalRead(hallSensorPin) == HIGH;
bool atNorth() {
  return digitalRead(hallSensorPin) == LOW;
}

// The magnet trigger point sits northOffsetDeg BEFORE true North, so we label it
// with a negative position. Step 0 is then exactly on the N mark.
void applyNorthOffset() {
  stepper.setCurrentPosition(-lround(northOffsetDeg * stepsPerDegree));
}

// Rotate forward until the LEADING edge of the magnet is found, then set the
// coordinate system so that 0 = true North.
void seekNorthAndZero() {
  stepper.setMaxSpeed(homingSpeed);
  stepper.setAcceleration(200);
  stepper.setSpeed(homingSpeed);     // forward

  while (atNorth()) stepper.runSpeed();   // step off the magnet if we start on it
  while (!atNorth()) stepper.runSpeed();  // roll on until the magnet is found
```

```
  applyNorthOffset();
}

// Drive from the magnet trigger point to true North (position 0), blocking.
void moveToTrueNorth() {
  stepper.setMaxSpeed(homingSpeed);
  stepper.setAcceleration(200);
  stepper.moveTo(0);
  while (stepper.distanceToGo() != 0) stepper.run();
}

void homeStepper() {
  Serial.println("Homing stepper motor to North...");
  seekNorthAndZero();
  Serial.println("Homing complete (on magnet edge).");
}

// Measure one full revolution between two identical leading edges, rotating in
// a single direction throughout. Sets stepsPerRevolution/stepsPerDegree.
void calibrateStepsPerRevolution() {
  Serial.println("Calibrating: measuring one magnet -> magnet revolution...");
  stepper.setMaxSpeed(homingSpeed);
  stepper.setAcceleration(200);

  // We're parked ON the magnet's leading edge, so start the count here.
  // setCurrentPosition() resets speed to 0, so setSpeed() must come after it.
  stepper.setCurrentPosition(0);
  stepper.setSpeed(homingSpeed);

  while (atNorth()) stepper.runSpeed();     // step off the magnet
  while (!atNorth()) stepper.runSpeed();    // next leading edge = one full turn

  long measured = labs(stepper.currentPosition());
  if (measured > 100) {
    stepsPerRevolution = measured;
    stepsPerDegree = (float)stepsPerRevolution / 360.0;
  } else {
    Serial.println("Calibration looked wrong; keeping default step count.");
  }

  // We finished on the magnet edge: label it with the offset so 0 = true North
  applyNorthOffset();

  Serial.print("Steps per revolution: ");
  Serial.println(stepsPerRevolution);
  Serial.print("Steps per degree: ");
  Serial.println(stepsPerDegree, 4);
  Serial.print("North offset (steps): ");
  Serial.println(lround(northOffsetDeg * stepsPerDegree));
}

// Quick re-zero on the magnet, then re-apply the current wind direction.
void rehome() {
  Serial.println("Auto re-home to North...");
  seekNorthAndZero();
  stepper.setMaxSpeed(400);
  stepper.setAcceleration(80);
  lastTargetWindDir = -1.0;   // force a fresh move to the latest direction
  Serial.println("Re-home complete.");
}

void initializeNeoPixel() {
  strip.begin();
  strip.setBrightness(neoBrightness);
  for (int i = 0; i < neoNumPixels; i++) {
    strip.setPixelColor(i, strip.Color(255, 255, 255));
  }
  strip.show();
  Serial.println("NeoPixel initialized and set to white.");
}

void setup() {
  Serial.begin(115200);
  Serial.println("Initializing system...");

  pinMode(hallSensorPin, INPUT_PULLUP);

  initializeNeoPixel();

  // Find the magnet edge
  homeStepper();

  Serial.println("On magnet. Waiting 5 seconds before calibration...");
  delay(5000);

  // Measure the real steps-per-revolution (ends on the magnet edge, offset applied)
  calibrateStepsPerRevolution();
  lastRehomeMs = millis();

  // Now nudge forward from the magnet onto TRUE North
  Serial.println("Moving to true North...");
  moveToTrueNorth();
  Serial.println("At true North. Check the pointer against the N mark.");
```

```
  // --- GLOBAL EASING CONFIGURATION ---
  stepper.setMaxSpeed(400);
  stepper.setAcceleration(80);
  // -----------------------------------

  delay(5000);

  // Self-test: sweep to South (180 deg) then back to North (0 deg)
  Serial.println("Test: moving to South (180 deg)...");
  stepper.moveTo(lround(180.0 * stepsPerDegree));
  while (stepper.distanceToGo() != 0) stepper.run();
  Serial.println("Reached South.");

  delay(5000);

  Serial.println("Returning to North (0 deg)...");
  stepper.moveTo(0);
  while (stepper.distanceToGo() != 0) stepper.run();
  Serial.println("Returned to North.");

  connectToWiFi();
  client.setServer(mqtt_server, mqtt_port);
  client.setCallback(mqttCallback);

  Serial.println("System ready.");
}

void loop() {
  if (WiFi.status() != WL_CONNECTED) {
    Serial.println("Wi-Fi connection lost. Reconnecting...");
    connectToWiFi();
  }

  if (!client.connected()) {
    reconnectToMQTT();
  }
  client.loop();

  // Periodic auto re-home: only when idle so we don't interrupt a move
  if (millis() - lastRehomeMs > rehomeIntervalMs && stepper.distanceToGo() == 0) {
    rehome();
    lastRehomeMs = millis();
  }

  moveStepperToWindDirection();
  stepper.run();
}
```